Agência de Relacionamento

Você foi contratado para criar uma aplicação de gerenciamento para a agência de relacionamento. A agência recebe cadastros de homens e mulheres e precisa de uma ferramenta para automatizar a tarefa de identificação de potenciais candidatos a encontros.

Os especialistas da agência desenvolveram um método que consideram infalível para encontrar os "pares". O método começa com uma seleção de potenciais pares baseada apenas nas áreas de interesse em comum, continuando com uma série de entrevistas com os potenciais candidatos a pares. A agência te contratou para desenvolver uma ferramenta que realiza a identificação inicial dos potenciais pares de forma automatizada, liberando os especialistas para tratarem especificamente do planejamento e realização das entrevistas.

A ferramenta deve utilizar como entrada os cadastros dos candidatos, que contêm:

a) Nome do candidato,
b) Estado em que reside (apenas a sigla),
c) Sexo (F/M),
d) Sexo do potencial companheiro (F/M/I)
e) Uma lista de códigos de áreas de interesse, escolhida das opções: 1: ESPORTES, 2: ARTES, 3: MÚSICA, 4: CINEMA, 5: TECNOLOGIA, 6: ANIMAIS, 7: GASTRONOMIA e 8: CIÊNCIAS

A ferramenta deve realizar as seguintes tarefas:

1. Inserir/atualizar o grau de interesse desejado. Por ex: 6.
2. Imprimir o número e a lista dos nomes de potenciais companheiros a formarem par a um dado candidato: os potenciais companheiros devem atender às seguintes restrições:
   a) Residirem no mesmo estado do candidato;
   b) Atenderem a opção sexual do candidato;
   c) Terem um grau de interesse em comum. (Definido pelo usuário)

   O grau de interesse em comum entre dois candidatos é definido em função das áreas de interesse em comum, atribuindo peso 1 para ESPORTES, MÚSICA e CINEMA, peso 2 para ARTES, ANIMAIS e GASTRONOMIA, e peso 3 para as demais áreas.
3. Imprimir id/nome dos K candidatos com maior número de potenciais companheiros;
4. Imprimir id/nome dos candidatos para os quais nenhum companheiro foi encontrado;
5. Imprimir id/nome de candidatos com interesse em uma determinada área;
6. Imprimir id/nome de candidatos que residem em um determinado estado.

Para implementar a aplicação deve-se criar um `TAD Candidato`. O `TAD Candidato` deve implementar uma estrutura de dados que, além dos campos descritos acima, também contém:

a) Identificador: um identificador interno para o Candidato
b) uma lista com os identificadores dos potenciais companheiros (inicialmente vazia)

O `TAD Candidato` deve implementar no mínimo as seguintes operações:

- `verificaPotencialCompanheiro(Candidato u, Candidato v)`: retorna verdadeiro ou falso se u e v atendem às restrições para serem potenciais companheiros;
- `inserePotencialCompanheiro(Candidato u, Candidato v)`: insere identificador de v na lista de candidatos de u. Note que você deve chamar uma função para obter o identificador de v e não acessá-lo diretamente;
- `numeroCompanheiros(Candidato u)`: retorna o número de possível companheiros
- `listaCompanheiros(Candidato u)`: retorna a lista dos potenciais companheiros de u, identificados pelo algoritmo;
- `temInteresse(Candidato u, int id)`: retorna verdadeiro se candidato u tem o id (tem outro candidato) na sua lista de interesses;
- `reside(Candidato u, String sigla)`: retorna verdadeiro se candidato u reside no estado fornecido.

Implemente também operações para criar um candidato no sistema, e para atualizar seus campos com as informações do cadastro. Você tem liberdade para decidir como estas operações serão implementadas (uma ou múltiplas funções, suas assinaturas, etc). Você também está livre para definir outras operações que julgar necessárias, e em particular, qualquer outra função/procedimento auxiliar (visível somente dentro do TAD) que seja necessária para garantir a estruturação adequada do TAD (vide comentário relativo a `inserePotencialCompanheiro`).

Além do `TAD Candidato`, crie também o `TAD ListaIdentificadores` que implementa uma lista com identificadores de potenciais candidatos. Você deve definir as operações necessárias para este TAD.

O restante do seu código deve implementar as 5 tarefas apresentadas acima utilizando as operações dos TADs. As informações dos candidatos devem ser lidas a partir de um arquivo de entrada `candidatos.txt`, com o seguinte formato:

```
N                  ──► número de candidatos cadastrados
****
nome
sigla_estado
sexo
sexo_interesse     ⟩  informações sobre 1 o candidato
número_areas
area_1
area_2
...
****
Nome
sigla_estado
sexo
sexo_interesse     ⟩  informações sobre o 2 o candidato
número_areas
area_1
area_2
...
```

O programa principal deve ser interativo, ou seja, deve permitir ao usuário da aplicação escolher qual tarefa deseja realizar. Em outras palavras, a interface deve permitir ao usuário escolher, interativamente, qual das 6 tarefas ele deseja realizar ou se ele deseja sair do sistema. Escolhida uma tarefa, a ferramenta deve solicitar que o usuário entre com as informações necessárias para a sua realização e imprimir os resultados na tela. A interface pode ser textual.

IMPORTANTE:

- Para facilitar o desenvolvimento e os testes da ferramenta, implemente um programa auxiliar para gerar automaticamente os dados dos candidatos e gravá-los no arquivo `candidatos.txt`. Os nomes podem ser lidos a partir de um arquivo dicionário com nomes de pessoas.
- Implemente as TADs usando lista simplesmente encadeada com tail e sentinelas.
- NÃO é permitido o uso de estruturas de dados já implementadas no Java, tais como: `ArrayList`, `LinkedList` ou similares. Todas as funções e as estruturas utilizadas deverão ser implementadas;

Entrega do Trabalho

O trabalho deve ser feito em grupos de dois alunos. Os programas devem ser redigidos em Java. Não será aceita nenhuma outra linguagem de programação.

O trabalho deve ser enviado pelo sigaa em um único arquivo ".zip".

O arquivo .zip deverá conter:

- Documento com Especificação de Requisitos, Arquitetura do Programa, ...
- Os arquivos-fonte dos diversos módulos que compõem o programa.
- Um arquivo LEIAME.TXT contendo a explicação de como utilizar o(s) programa(s).
- Arquivos RELATORIO-nome.TXT de cada membro do grupo. O tema nome deve identificar o membro do grupo ao qual se refere o relatório. Estes arquivos devem conter uma tabela de registro de trabalho organizada como a seguir:

  Data ; Horas Trabalhadas ; Tipo Tarefa ; Descrição da Tarefa Realizada

  - Na descrição da tarefa redija uma explicação breve sobre o que o componente do grupo fez. Esta descrição deve estar de acordo com o Tipo Tarefa. Cada Tipo Tarefa identifica uma natureza de atividade que deverá ser discriminada explicitamente, mesmo que, durante uma mesma sessão de trabalho tenham sido realizadas diversas tarefas.

Os tipos de tarefa são:

- Estudar
- Especificar os módulos/pacote/classes
- Especificar as funções/métodos
- Revisar especificações
- Projetar
- Revisar projetos
- Codificar módulo
- Revisar código do módulo
- Realizar os testes
- Diagnosticar e corrigir os problemas encontrados

- Data de entrega: 14/10/2026

BOM TRABALHO!